

Université Hassan 1^{er}
École Nationale des Sciences Appliquées — Berrechid
Filière : Systèmes d'Information et Big Data (ISIBD)

RAPPORT DE PROJET
Module : Ingénierie et Maquette Numérique

CrowdFlow VR

*Simulation de Gestion de Foule par Intelligence
Artificielle en Réalité Virtuelle*



MOTEUR UNITY 6.4 · META QUEST 2 · OPENXR · PYTHON

Encadré par : Prof. Hamid Hrimech

Année universitaire : 2025 / 2026

RÉALISÉ PAR

Maroua Lhassouani — Imane Hamraoui — Hoda Mounaouir
Maryem Mouslim — Assil Nagda — Hasnaa Saadi

ANNÉE UNIVERSITAIRE 2025 - 2026

Remerciements

Au terme de ce projet, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation.

Nous adressons nos remerciements les plus sincères à notre encadrant, **Professeur Hamid Hrimech**, pour la qualité de son encadrement, ses conseils avisés, sa disponibilité et la confiance qu'il nous a accordée tout au long de ce travail. Ses orientations ont été déterminantes dans la conduite et l'aboutissement de ce projet.

Nous remercions également l'ensemble du corps professoral et administratif de l'**École Nationale des Sciences Appliquées de Berrechid**, et plus particulièrement les enseignants de la filière **Systèmes d'Information et Big Data**, pour la formation de qualité qu'ils nous ont dispensée et qui a constitué le socle de ce projet.

Enfin, nous exprimons toute notre reconnaissance à nos familles et à nos camarades pour leur soutien constant et leurs encouragements, ainsi qu'aux membres de l'équipe pour l'esprit de collaboration et la complémentarité qui ont animé ce travail collectif.

À toutes et à tous, nous adressons nos remerciements les plus chaleureux.

Résumé

La gestion des flux de foule dans les espaces publics à forte affluence représente un enjeu majeur de sécurité, où une densité mal anticipée peut conduire à des situations critiques. Ce projet, intitulé **CrowdFlow VR**, propose un environnement de simulation immersive de gestion de foule au sein d'une gare ferroviaire reconstituée en trois dimensions.

Le système repose sur une architecture distribuée en quatre couches. Une foule d'agents virtuels autonomes navigue de manière réaliste grâce au système **Nav-Mesh** du moteur **Unity**, tandis que des modèles comportementaux — le modèle de force sociale de Helbing et la séparation de type Boids — produisent une dynamique collective crédible. Un moteur d'**intelligence artificielle**, fondé sur un algorithme **IsolationForest** entraîné sur des données piétonnes réelles (jeux de données ETH/UCY), détecte en temps réel les anomalies de densité et de vitesse. L'ensemble des métriques est restitué sur un **tableau de bord Big Data** temps réel développé avec Streamlit, et l'expérience est rendue immersive grâce à une interface de **réalité virtuelle** compatible Meta Quest 2 (OpenXR), complétée par un mode PC.

Le projet, mené collaborativement par six contributrices via un workflow Git structuré, démontre l'intégration cohérente de la simulation comportementale, de l'apprentissage automatique, du Big Data et de la réalité virtuelle au sein d'un même système fonctionnel.

Mots-clés : Simulation de foule, Réalité Virtuelle, Intelligence Artificielle, Détection d'anomalies, NavMesh, Unity, Big Data, Meta Quest 2, OpenXR.

Abstract

Managing crowd flows in high-traffic public spaces is a major safety concern, where poorly anticipated density can lead to critical situations. This project, entitled **CrowdFlow VR**, introduces an immersive crowd-management simulation environment set within a three-dimensional reconstruction of a railway station.

The system relies on a four-layer distributed architecture. A crowd of autonomous virtual agents navigates realistically through the **NavMesh** system of the **Unity** engine, while behavioural models — Helbing's social force model and Boids-style separation — produce a credible collective dynamic. An **artificial intelligence** engine, based on an **IsolationForest** algorithm trained on real pedestrian data (ETH/UCY datasets), detects density and speed anomalies in real time. All metrics are streamed to a real-time **Big Data dashboard** built with Streamlit, and the experience is made immersive through a **virtual reality** interface compatible with the Meta Quest 2 (OpenXR), complemented by a desktop mode.

Carried out collaboratively by six contributors through a structured Git workflow, the project demonstrates the coherent integration of behavioural simulation, machine learning, Big Data and virtual reality within a single functional system.

Keywords: Crowd simulation, Virtual Reality, Artificial Intelligence, Anomaly detection, NavMesh, Unity, Big Data, Meta Quest 2, OpenXR.

Table des matières

Introduction générale

Chapitre 1 — Présentation générale du projet

1.1 Contexte général 1.2 Problématique 1.3 Objectifs 1.4 Cas d'usage 1.5 Périmètre fonctionnel

Chapitre 2 — Outils et technologies utilisés

2.1 Moteur Unity 2.2 Python & scikit-learn 2.3 Réalité virtuelle 2.4 Streamlit 2.5 Git/GitHub

Chapitre 3 — Architecture du système

3.1 Architecture en quatre couches 3.2 Pile technologique 3.3 Flux de données temps réel

Chapitre 4 — Contributions détaillées de l'équipe

4.1 Maroua 4.2 Imane 4.3 Hoda 4.4 Maryem 4.5 Assil 4.6 Hasnaa

Chapitre 5 — Intelligence Artificielle

5.1 Détection d'anomalies 5.2 Navigation NavMesh 5.3 Dynamique collective

Chapitre 6 — Big Data et tableau de bord

Chapitre 7 — Architecture et interaction en réalité virtuelle

Chapitre 8 — Gestion de projet et organisation GitHub

Chapitre 9 — Résultats, tests et performances

Conclusion générale et perspectives

Références bibliographiques

Liste des figures

- Figure 1.1 — Vue d'ensemble de la gare CrowdFlow VR dans l'éditeur Unity
- Figure 3.1 — Schéma de l'architecture en quatre couches
- Figure 4.1 — Table de répartition de l'équipe (README du dépôt)
- Figure 4.2 — Page d'accueil du dépôt GitHub
- Figure 4.3 — Branches de fonctionnalités et Pull Requests
- Figure 4.4 — Arborescence des scripts par domaine
- Figure 4.5 — Exploration des données dans Jupyter
- Figure 4.6 — Entraînement de l'IsolationForest et résultats
- Figure 4.7 — Zone des guichets de billets
- Figure 4.8 — Distributeur automatique et mobilier
- Figure 4.9 — Dossier des prefabs du projet
- Figure 4.10 — Vue de dessus : navigation des agents
- Figure 4.11 — Hiérarchie Unity des agents et points
- Figure 4.12 — En-tête du tableau de bord temps réel
- Figure 4.13 — Heatmap de densité et timeline
- Figure 4.14 — États des agents et distribution des vitesses
- Figure 4.15 — Vue VR : zone de danger et barrière
- Figure 4.16 — Panneau de sélection des entrées VR
- Figure 4.17 — Hiérarchie intégrée (NetworkManager, NetworkSync)
- Figure 4.18 — Hiérarchie : PanicZone, XR Origin, prefabs
- Figure 4.19 — Gros plan sur la séparation entre agents
- Figure 5.1 — Exploration des données réelles ETH
- Figure 5.2 — Trajectoires réelles de piétons
- Figure 6.1 — Sortie du serveur d'inférence en fonctionnement
- Figure 7.1 — Rendu de la scène avec personnages variés
- Figure 8.1 — Organisation des scripts dans le projet Unity
- Figure 9.1 — Lancement du serveur ML en attente

INTRODUCTION

Introduction générale

Les espaces publics à forte affluence — gares, aéroports, stades, centres commerciaux — accueillent quotidiennement des flux importants de personnes. La maîtrise de ces flux constitue un enjeu de sécurité de premier plan : une densité mal anticipée, conjuguée à un événement déclencheur, peut provoquer des mouvements de foule aux conséquences parfois dramatiques.

Anticiper, visualiser et comprendre le comportement d'une foule est donc essentiel pour les gestionnaires d'infrastructures. Or, l'expérimentation grandeur nature de scénarios critiques est impossible pour des raisons évidentes de sécurité. La **simulation numérique** s'impose alors comme l'outil privilégié pour étudier ces phénomènes, à condition de reproduire fidèlement à la fois les déplacements individuels et la dynamique collective.

C'est dans ce cadre que s'inscrit le projet **CrowdFlow VR**, réalisé dans le cadre du module *Ingénierie et Maquette Numérique*. Il vise à concevoir un environnement de simulation immersive de gestion de foule au sein d'une gare ferroviaire virtuelle, en combinant trois disciplines complémentaires : la simulation comportementale d'agents autonomes, l'intelligence artificielle pour la détection d'anomalies, et la réalité virtuelle comme interface d'observation et d'action.

Ce rapport présente l'ensemble du travail réalisé. Le **chapitre 1** expose la présentation générale du projet ; le **chapitre 2** détaille les outils et technologies employés ; le **chapitre 3** décrit l'architecture du système ; le **chapitre 4** valorise les contributions individuelles de chaque membre de l'équipe ; les **chapitres 5 à 7** approfondissent respectivement l'intelligence artificielle, le Big Data et la réalité virtuelle ; le **chapitre 8** traite de la gestion de projet ; et le **chapitre 9** présente les résultats et tests. Une conclusion dresse le bilan et ouvre des perspectives.

CHAPITRE 1

Présentation générale du projet

1.1 Contexte général

La gestion des flux de foule dans les lieux à forte densité est un domaine d'étude pluridisciplinaire mobilisant la physique des foules, la psychologie comportementale et l'ingénierie de la sécurité. Les gares ferroviaires en constituent un cas emblématique : elles concentrent, sur des plages horaires courtes, des centaines de voyageurs aux trajectoires variées (accès aux quais, guichets, zones d'attente), créant des configurations de densité fluctuantes et potentiellement critiques.

Le projet **CrowdFlow VR** propose de modéliser et de simuler ces dynamiques dans un environnement virtuel reproduisant fidèlement une gare, afin d'observer le comportement de la foule, de détecter automatiquement les situations à risque et d'expérimenter des mesures d'intervention.

1.2 Problématique

La problématique centrale du projet peut se formuler ainsi : **comment anticiper et visualiser, en temps réel, l'apparition de situations critiques au sein d'une foule, tout en permettant à un opérateur d'intervenir physiquement dans l'environnement simulé ?**

Répondre à cette question suppose de relever plusieurs défis techniques : simuler un grand nombre d'agents autonomes au comportement réaliste, détecter automatiquement les anomalies sans supervision humaine, restituer les indicateurs sous une forme exploitable, et offrir une interface d'interaction intuitive.

1.3 Objectifs

Le projet poursuit les objectifs suivants :

- Simuler une foule d'agents autonomes naviguant de manière réaliste à l'aide du système NavMesh d'Unity ;
- Reproduire des comportements collectifs crédibles (évitement, séparation, contagion de la panique, évacuation) à partir de modèles scientifiques reconnus ;
- Détecter automatiquement les anomalies de densité et de vitesse grâce à un modèle d'apprentissage automatique entraîné sur des données piétonnes réelles ;
- Restituer l'ensemble des métriques sur un tableau de bord Big Data temps réel ;

- Offrir une expérience immersive en réalité virtuelle sur Meta Quest 2, complétée par un mode PC, permettant l'observation et le placement interactif de barrières.

1.4 Cas d'usage : la gare ferroviaire

La scène modélise une gare complète : guichets de billets, panneaux d'horaires, tourniquets, distributeurs automatiques, zones d'attente meublées, caméras de surveillance et personnages variés (voyageurs, enfants, personnes âgées, agents de police). Les piétons apparaissent depuis des points d'entrée (*SpawnPoints*) et se dirigent vers des points de sortie (*ExitPoints*) ou vers une sortie d'urgence lorsqu'une évacuation est déclenchée.



Figure 1.1 — Vue d'ensemble de la gare CrowdFlow VR dans l'éditeur Unity : zones d'attente, distributeurs, tourniquet central, guichets en arrière-plan et personnages 3D.

1.5 Périmètre fonctionnel

Le système livré couvre l'intégralité de la chaîne, depuis la génération des agents jusqu'à la restitution analytique, en passant par la détection d'anomalies et l'immersion. Le dépôt public du projet — github.com/maryemmoulim/crowdflowvr — regroupe le code source, les assets et la documentation (Unity 6.4, neuf branches, soixante-et-un commits).

CHAPITRE 2

Outils et technologies utilisés

La réalisation de CrowdFlow VR a mobilisé un ensemble cohérent d'outils couvrant la modélisation 3D, l'apprentissage automatique, la réalité virtuelle, la visualisation de données et la gestion de version.

2.1 Le moteur Unity

Unity (version 6000.4.0f1, dite Unity 6.4) est le moteur de jeu temps réel qui sert de socle au projet. Il fournit l'éditeur de scène 3D, le système de navigation **Nav-Mesh** pour le déplacement des agents, le pipeline de rendu **Universal Render Pipeline (URP 17.4)**, ainsi que le langage de script **C#**. Unity gère également l'intégration des assets, des prefabs, des matériaux et de l'éclairage de la gare.

2.2 Python et scikit-learn

Le moteur d'intelligence artificielle est développé en **Python 3.11**. La bibliothèque **scikit-learn** fournit l'algorithme **IsolationForest** utilisé pour la détection d'anomalies, tandis que **NumPy** et **pandas** assurent la manipulation des données et **Matplotlib** la visualisation exploratoire au sein de notebooks **Jupyter**. La communication temps réel avec Unity s'effectue via des *sockets* UDP.

2.3 La réalité virtuelle : OpenXR et Meta Quest 2

L'immersion repose sur le casque **Meta Quest 2** et le standard ouvert **OpenXR (1.16)**, intégrés à Unity via le **XR Interaction Toolkit (3.5)** et le plug-in **XR Oculus (4.5)**. Le nouveau **Input System (1.19)** d'Unity gère les entrées clavier et manettes, et **Netcode for GameObjects** assure la synchronisation réseau du mode multijoueur.

2.4 Streamlit

Streamlit est le framework Python retenu pour construire le tableau de bord analytique. Il permet de transformer des scripts Python en applications web interactives, ici exploitées pour afficher en temps réel les métriques de foule, les heatmaps de densité et les graphiques d'évolution.

2.5 Git et GitHub

Le projet est versionné avec **Git** et hébergé sur **GitHub**. Un workflow par branches de fonctionnalités et Pull Requests a été adopté afin de permettre le travail parallèle des six membres de l'équipe sans conflit.

Domaine	Outils et versions
Moteur 3D	Unity 6000.4.0f1 (6.4) · Universal Render Pipeline 17.4
Réalité virtuelle	XR Interaction Toolkit 3.5 · OpenXR 1.16 · XR Oculus 4.5 · Meta Quest 2
Entrées / Réseau	Input System 1.19 · Netcode for GameObjects
Intelligence artificielle	Python 3.11 · scikit-learn 1.8 · NumPy 2.4 · pandas 3.0 · Jupyter
Big Data / Visualisation	Streamlit · pandas · Matplotlib
Gestion de version	Git · GitHub (branches feature + Pull Requests)

Tableau 2.1 — Synthèse de la pile technologique du projet.

CHAPITRE 3

Architecture du système

3.1 Une architecture en quatre couches

CrowdFlow VR repose sur une architecture distribuée organisée en quatre couches communicantes. La simulation Unity et le moteur d'intelligence artificielle Python s'exécutent en parallèle et échangent en continu, tandis que le tableau de bord consomme les journaux produits par la simulation.

Couche	Rôle	Technologie	Responsable
1. Collecte	Capture des positions et vitesses, envoi vers le moteur IA	Unity C# · UDP (5005)	Maroua
2. Moteur ML	Inférence temps réel, densité, détection de danger	Python · scikit-learn	Maroua
3. Simulation	Agents NavMesh, états, forces sociales, panique	Unity 6.4 · Nav-Mesh · URP	Imane · Hasnaa · Assil
4. Interface	Immersion VR / mode PC + tableau de bord	OpenXR · Streamlit	Maryem · Hoda

Tableau 3.1 — Les quatre couches fonctionnelles et leurs responsables.

3.2 Flux de données temps réel

Toutes les 500 millisecondes, le client Unity sérialise l'état de la foule (positions et vitesses) au format JSON et l'émet en UDP vers le serveur Python. Celui-ci calcule la densité et un indicateur de danger, puis renvoie sa réponse, exploitée par la simulation pour basculer les agents en panique. En parallèle, un module de journalisation écrit l'état complet dans un fichier CSV horodaté que le tableau de bord lit en continu.

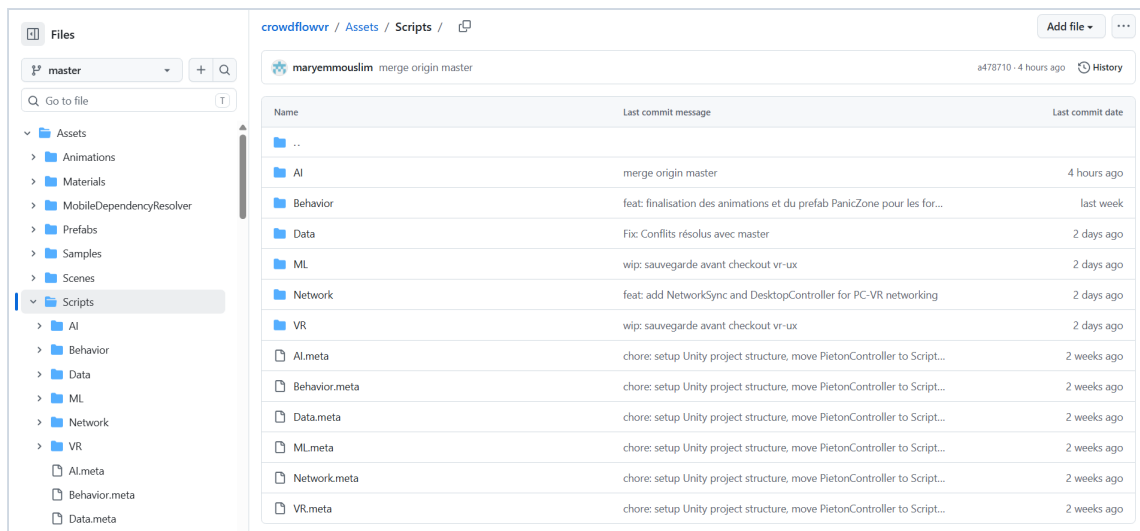


Figure 3.1 — Organisation modulaire du code reflétant l'architecture : un répertoire par domaine fonctionnel (AI, Behavior, Data, ML, Network, VR).

```
// Chaîne de traitement de bout en bout
Unity (agents) --[JSON / UDP:5005]--> Serveur ML Python --> danger / densité
|
|--[CSV session_*.csv / 500 ms]--> Dashboard Streamlit <----+
```

Schéma logique de l'échange temps réel entre la simulation, le moteur d'inférence et le tableau de bord.

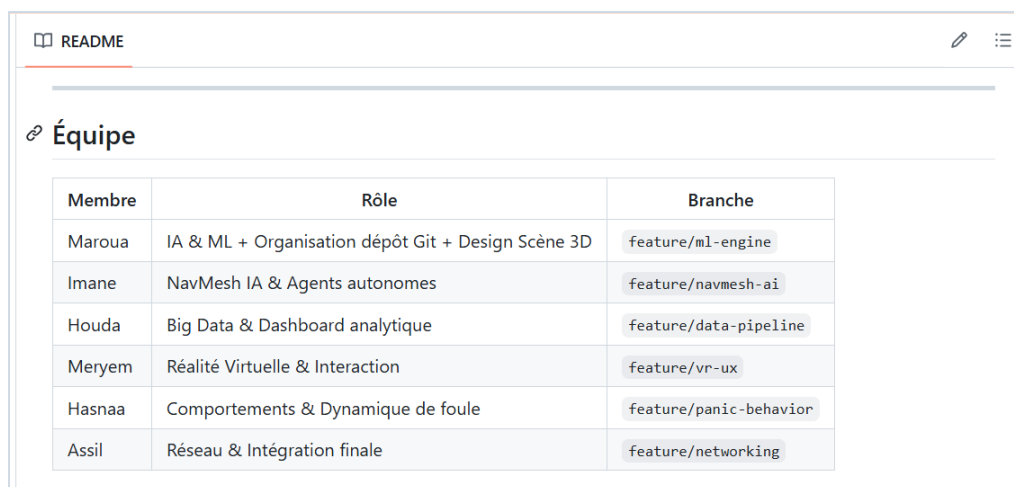
CHAPITRE 4

Contributions détaillées de l'équipe

Le projet a été mené par six contributrices, chacune responsable d'un pilier technique. Le tableau ci-dessous synthétise la répartition ; les sections suivantes détaillent les livrables réels de chaque membre.

Membre	Pilier	Branche Git
Maroua Lhassouani	IA / ML · Organisation Git · Design Scène 3D	<code>feature/ml-engine</code>
Imane Hamraoui	NavMesh IA & Agents autonomes	<code>feature/navmesh-ai</code>
Hoda Mounaouir	Big Data & Dashboard analytique	<code>feature/data-pipeline</code>
Maryem Mouslim	Réalité Virtuelle & Interaction	<code>feature/vr-ux</code>
Assil Nagda	Réseau & Intégration finale	<code>feature/networking</code>
Hasnaa Saadi	Comportements & Dynamique de foule	<code>feature/panic-behavior</code>

Tableau 4.1 — Répartition des rôles au sein de l'équipe.



The screenshot shows a README file with a table titled "Équipe". The table has three columns: "Membre", "Rôle", and "Branche". The data is as follows:

Membre	Rôle	Branche
Maroua	IA & ML + Organisation dépôt Git + Design Scène 3D	<code>feature/ml-engine</code>
Imane	NavMesh IA & Agents autonomes	<code>feature/navmesh-ai</code>
Houda	Big Data & Dashboard analytique	<code>feature/data-pipeline</code>
Meryem	Réalité Virtuelle & Interaction	<code>feature/vr-ux</code>
Hasnaa	Comportements & Dynamique de foule	<code>feature/panic-behavior</code>
Assil	Réseau & Intégration finale	<code>feature/networking</code>

Figure 4.1 — Table de répartition de l'équipe documentée dans le fichier README du dépôt.

4.1 — Maroua Lhassouani

Intelligence Artificielle & Machine Learning · Organisation du dépôt Git · Conception de la scène 3D

branche : `feature/ml-engine`

Maroua a porté trois responsabilités transverses et structurantes : la chaîne d'intelligence artificielle, l'organisation technique du dépôt Git partagé, et la conception complète de la scène 3D.

a) Organisation du dépôt Git

Elle a initialisé le dépôt, rédigé le fichier `.gitignore` adapté à Unity, défini la structure de dossiers attribuant à chaque membre son sous-répertoire de scripts, et créé les branches de fonctionnalités permettant un travail parallèle : `master`, `develop` et les branches *feature* de l'équipe.

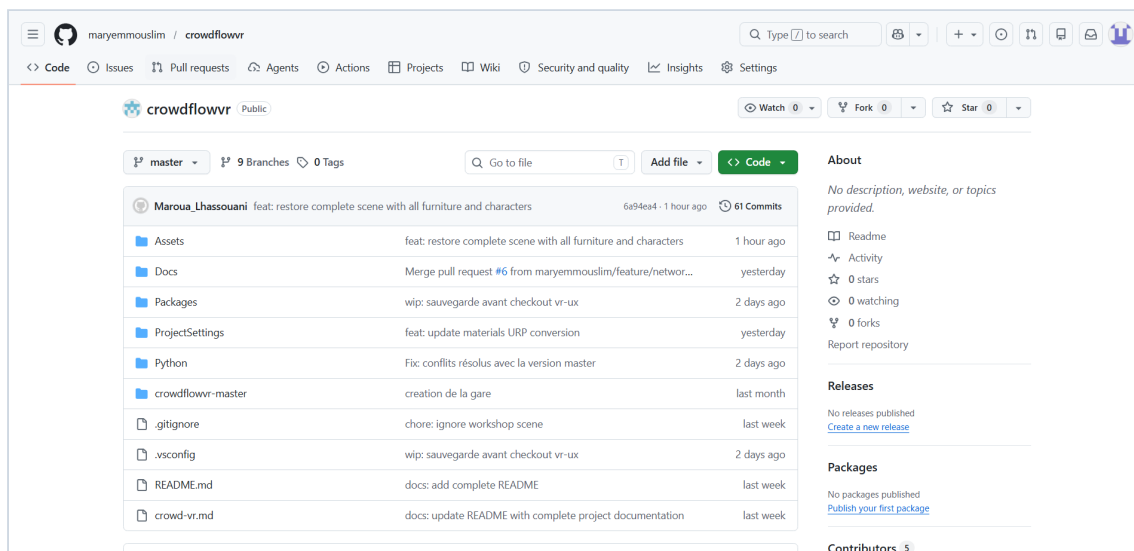


Figure 4.2 — Page d'accueil du dépôt GitHub : organisation en dossiers Assets, Python, Docs et ProjectSettings.

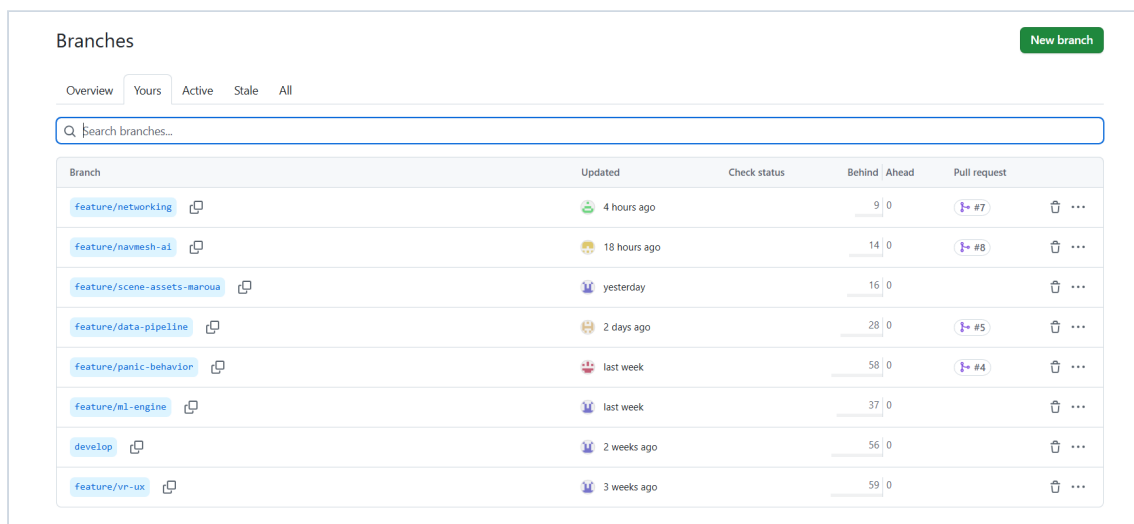


Figure 4.3 — Les branches de fonctionnalités et leurs Pull Requests, reflétant le flux de collaboration mis en place.

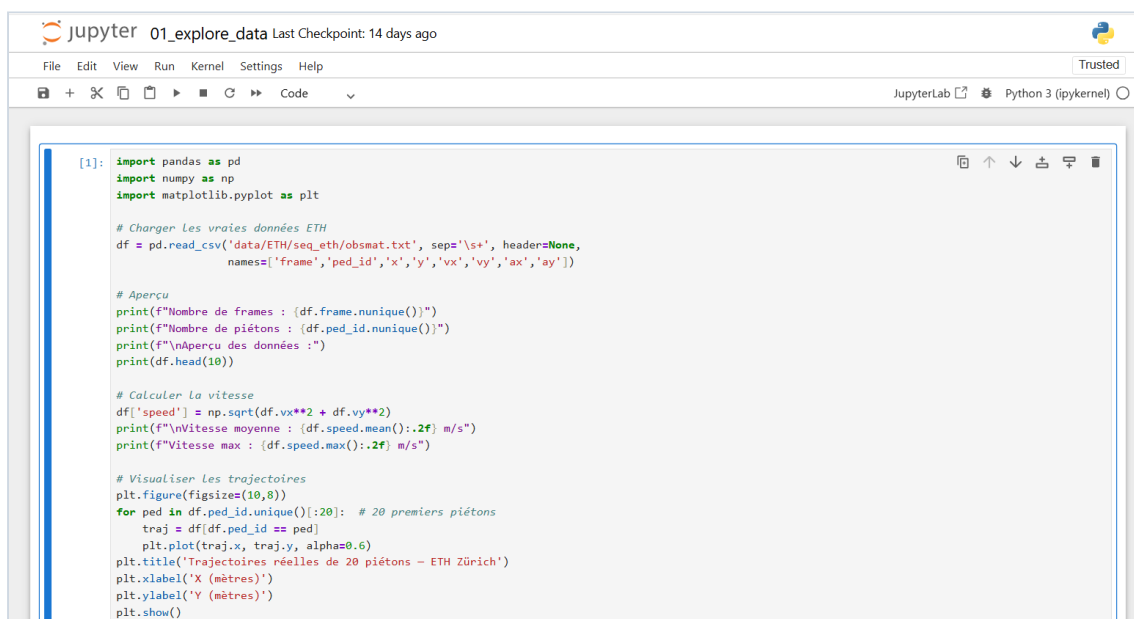
b) Pipeline de Machine Learning sur données réelles

Maroua a construit la chaîne ML à partir du jeu de données piétonnes de référence **ETH/UCY** (issu d'OpenTraj, publications ICCV 2007 et 2009). Après exploration des trajectoires, elle a extrait quatre caractéristiques par frame — nombre

d'agents, vitesse moyenne, écart-type et 90^e percentile — normalisées via `StandardScaler`, puis entraîné un modèle **IsolationForest**, sauvegardé dans `model.pkl`.

```
# Extraction des caractéristiques par frame puis entraînement
g = d.groupby('frame').agg(
    nb=('ped_id', 'count'), mu=('speed', 'mean'),
    sd=('speed', 'std'), p90=('speed', lambda x: np.percentile(x, 90))
)
X_sc = StandardScaler().fit_transform(X)
model = IsolationForest(contamination=0.05, n_estimators=100, random_state=42)
model.fit(X_sc)
# Modèle entraîné sur 4534 frames réelles – 227 anomalies détectées (5,0 %)
```

Extrait du notebook d'entraînement de l'IsolationForest (apprentissage non supervisé).



```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Charger Les vraies données ETH
df = pd.read_csv('data/ETH/seq_eth/obsmat.txt', sep='\\s+', header=None,
                names=['frame', 'ped_id', 'x', 'y', 'vx', 'vy', 'ax', 'ay'])

# Aperçu
print(f"Nombre de frames : {df.frame.nunique()}")
print(f"Nombre de piétons : {df.ped_id.nunique()}")
print(f"\\nAperçu des données :")
print(df.head(10))

# Calculer la vitesse
df['speed'] = np.sqrt(df.vx**2 + df.vy**2)
print(f"\\nVitesse moyenne : {df.speed.mean():.2f} m/s")
print(f"Vitesse max : {df.speed.max():.2f} m/s")

# Visualiser les trajectoires
plt.figure(figsize=(10,8))
for ped in df.ped_id.unique()[:20]: # 20 premiers piétons
    traj = df[df.ped_id == ped]
    plt.plot(traj.x, traj.y, alpha=0.6)
plt.title('Trajectoires réelles de 20 piétons – ETH Zürich')
plt.xlabel('X (mètres)')
plt.ylabel('Y (mètres)')
plt.show()
```

Figure 4.5 — Chargement et exploration des données réelles ETH dans Jupyter (calcul des vitesses, tracé des trajectoires).


```

frames = []
for path in scenes:
    d = pd.read_csv(path, sep='\s+', header=None,
                    names=['frame', 'ped_id', 'x', 'y', 'vx', 'vy', 'ax', 'ay'])
    d['speed'] = np.sqrt(d.vx**2 + d.vy**2)
    g = d.groupby('frame').agg(
        nb=('ped_id', 'count'),
        mu=('speed', 'mean'),
        sd=('speed', 'std'),
        p90=('speed', lambda x: np.percentile(x, 90))
    ).fillna(0)
    frames.append(g)

X = pd.concat(frames)[['nb', 'mu', 'sd', 'p90']].values

# Normaliser
scaler = StandardScaler()
X_sc = scaler.fit_transform(X)

# Entraîner IsolationForest
model = IsolationForest(contamination=0.05, n_estimators=100, random_state=42)
model.fit(X_sc)

# Sauvegarder
pickle.dump({'model': model, 'scaler': scaler}, open('model.pkl', 'wb'))

anomalies = (model.predict(X_sc) == -1).sum()
print(f"✅ Modèle entraîné sur {len(X)} frames réelles")
print(f"🚨 Anomalies détectées : {anomalies} ({anomalies/len(X)*100:.1f}%)")

✅ Modèle entraîné sur 4534 frames réelles
🚨 Anomalies détectées : 227 (5.0%)

```

Figure 4.6 — Entraînement de l'IsolationForest et résultat : 4534 frames, 227 anomalies (5,0 %).

c) Serveur d'inférence UDP et client Unity

Côté Python, le script `infer_server.py` écoute sur le port UDP 5005, reçoit l'état de la foule, calcule densité et danger, puis répond en temps réel. Côté Unity, le script C# `MLClient.cs` collecte les positions des agents et dialogue avec le serveur toutes les 500 ms, exposant un booléen `dangerDetected`.

```

// MLClient.cs – émission UDP de l'état de la foule (toutes les 500 ms)
udp.Send(data, data.Length, mlServerIP, mlServerPort); // 127.0.0.1:5005
dangerDetected = result.danger; density = result.density;

```

d) Conception de la scène 3D

Maroua a importé et assemblé plus de dix packs de l'Asset Store (meublier de bureau, caméras de surveillance, personnages, poubelles low-poly, distributeurs...). Elle a créé manuellement les prefabs propres au projet — `Guichet_Billets`, `Panneau_Horaires`, `Barriere_Tourniquet` — puis réalisé le placement complet de la gare.

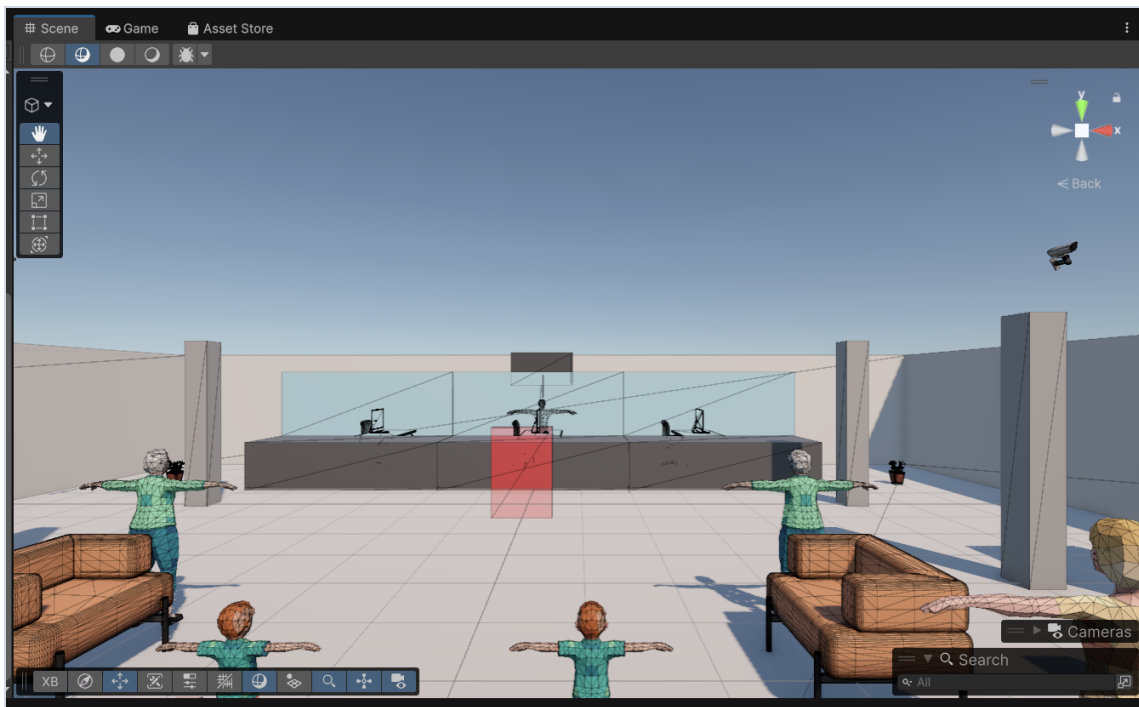


Figure 4.7 — Zone des guichets de billets avec mobilier et personnages 3D.



Figure 4.8 — Distributeur automatique et poubelle low-poly intégrés le long des colonnes de la gare.

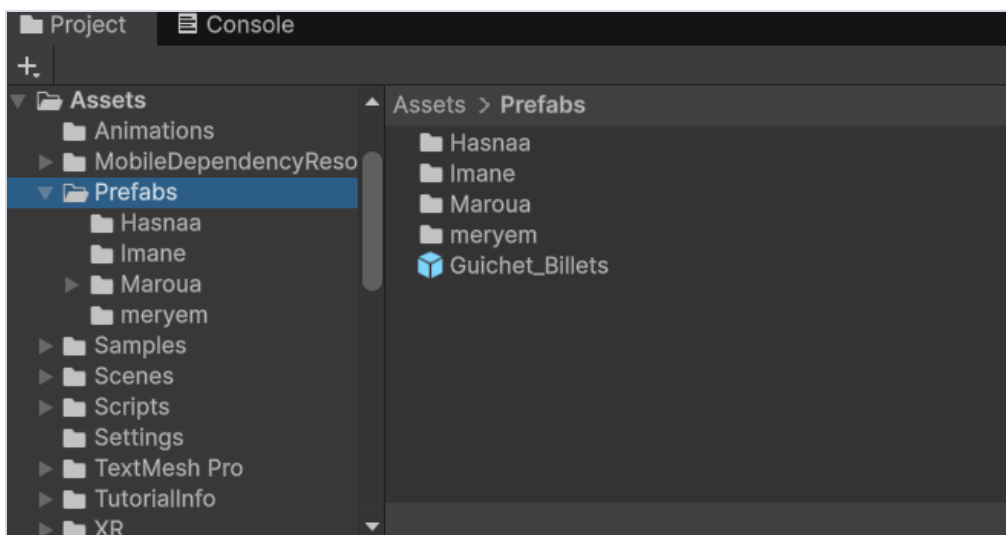


Figure 4.9 — Dossier des prefabs du projet : sous-dossiers par membre et prefab `Guichet_Billets`.

4.2 — Imane Hamraoui

Navigation NavMesh & Agents autonomes

branche : `feature/navmesh-ai`

Imane a conçu le cœur de la simulation de mouvement : la navigation autonome des piétons. Après le *bake* complet du NavMesh sur la gare, elle a développé deux scripts centraux et remplacé les capsules de test par de véritables personnages 3D.

a) Machine à états des piétons

Chaque agent embarque une machine à états à quatre comportements — `Normal`, `Dense`, `Panic`, `Evacuation` — chacun associé à une vitesse propre. L'agent choisit dynamiquement ses destinations, ou se dirige vers la sortie d'urgence lors d'une évacuation.

```
// CrowdAgent.cs – états et vitesses associées
public enum AgentState { Normal, Dense, Panic, Evacuation }
public float normalSpeed = 1.4f, denseSpeed = 0.8f,
    panicSpeed = 3.5f, evacuationSpeed = 2.0f;
```

b) Génération dynamique des agents

Le *spawner* instancie les piétons depuis plusieurs points de départ, valide chaque position via `NavMesh.SamplePosition`, et surveille en continu l'état `dangerDetected` du moteur ML pour basculer toute la foule en panique. Il expose la méthode `TriggerEvacuation()` utilisée lors de l'intégration finale.

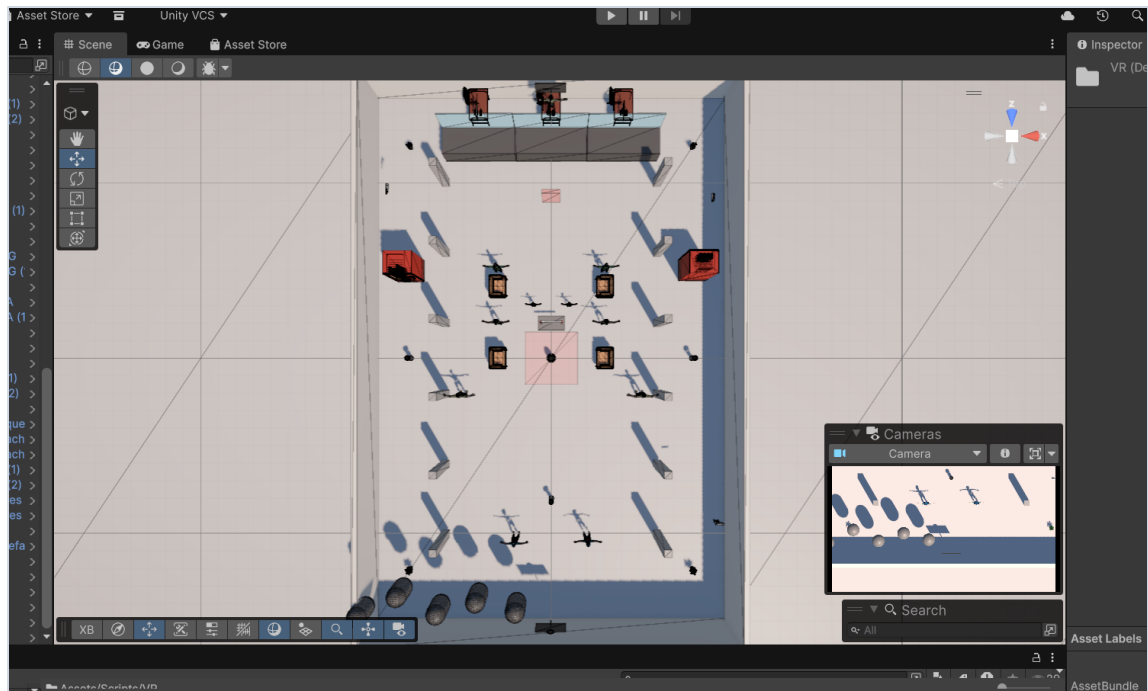


Figure 4.10 — *Vue de dessus de la scène : agents 3D en cours de navigation NavMesh vers les zones de la gare.*

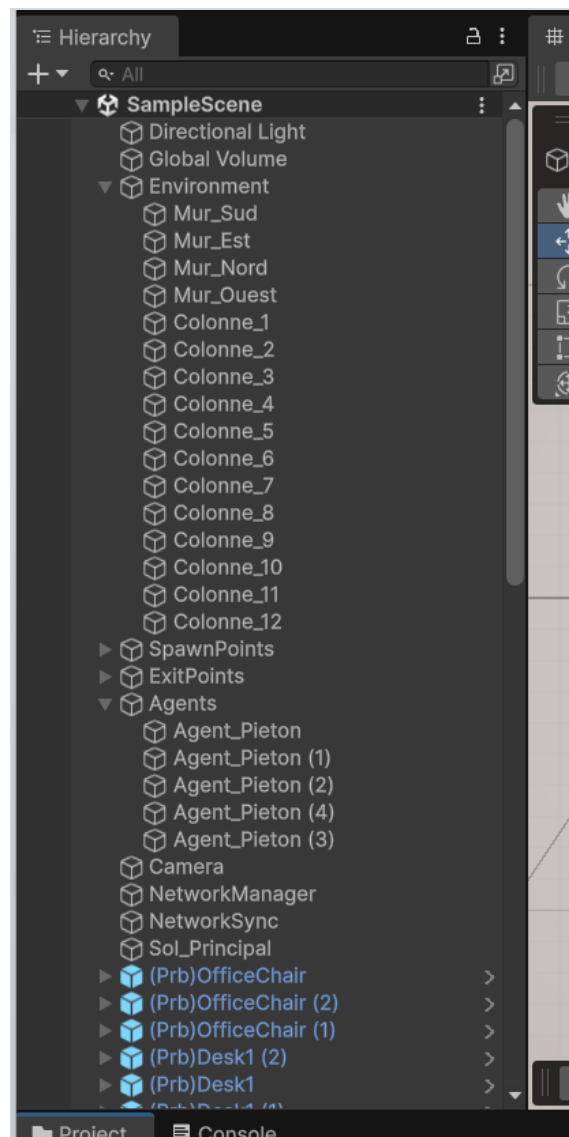


Figure 4.11 — Hiérarchie Unity : conteneurs *SpawnPoints*, *ExitPoints* et *Agents* instanciés.

Paramètre NavMeshAgent	Valeur	Test réalisé	Résultat
Radius / Height	0.3 / 1.8	Navigation (1 agent)	0 erreur
Speed / Angular Speed	1.4 / 240	Génération (10 agents)	0 erreur
Acceleration	8	Évacuation (touche E)	Tous vers la sortie
Stopping Distance	0.4	Performance (50 agents)	≈ 19-27 FPS

Tableau 4.2 — Paramètres du prefab CrowdAgent et résultats des tests de navigation.

4.3 — Hoda Mounaouir

Big Data & Tableau de bord analytique temps réel

branche : feature/data-pipeline

Hoda a mis en place la chaîne Big Data du projet : la capture continue des données de simulation, le calcul des métriques de foule, et un tableau de bord interactif restituant l'état de la gare en temps réel.

a) Journalisation temps réel

Le module `DataLogger.cs` écrit, toutes les 500 ms, l'état de chaque agent dans un fichier CSV horodaté (`session_HHmmss.csv`). Le format de colonnes (`frame, ped_id, x, z, speed, state`) est aligné sur le lecteur du dashboard, avec une écriture sécurisée évitant les conflits avec Python.

b) Calcul des métriques

Le composant `MetricsManager.cs` surveille la densité locale autour de chaque agent (rayon de 3 m) et déclenche une alerte dès qu'un regroupement critique (seuil de 8 agents) est détecté, en localisant la zone concernée.

c) Tableau de bord Streamlit

Le dashboard (`localhost:8501`) bascule automatiquement entre un mode LIVE (lecture du CSV Unity) et un mode simulation. Il affiche les métriques clés, une heatmap de densité, la courbe du nombre d'agents, la répartition des états et la distribution des vitesses.

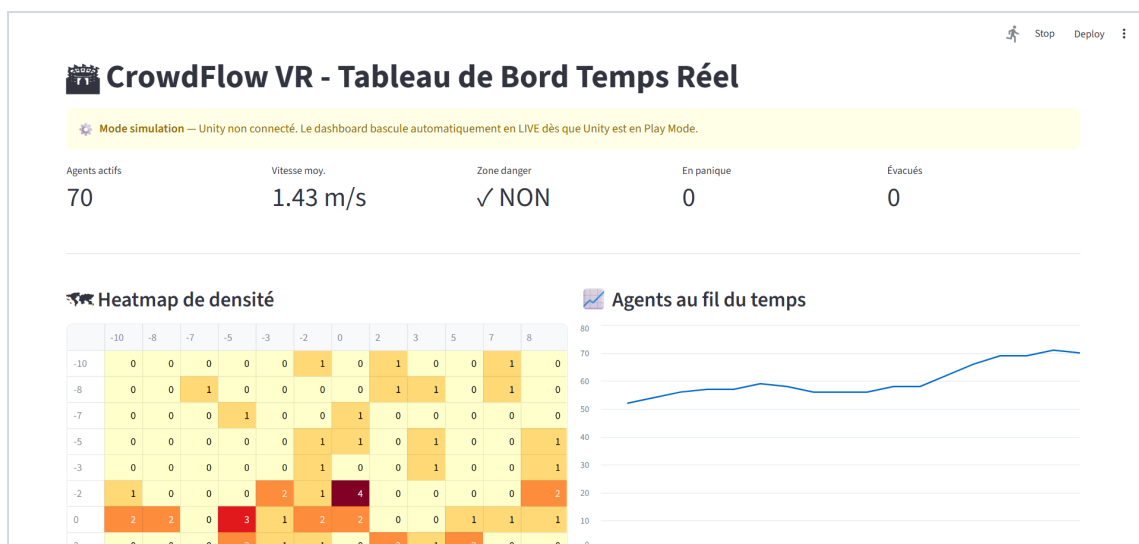


Figure 4.12 — En-tête du tableau de bord : 70 agents actifs, vitesse moyenne 1,43 m/s, zone de danger « NON ».

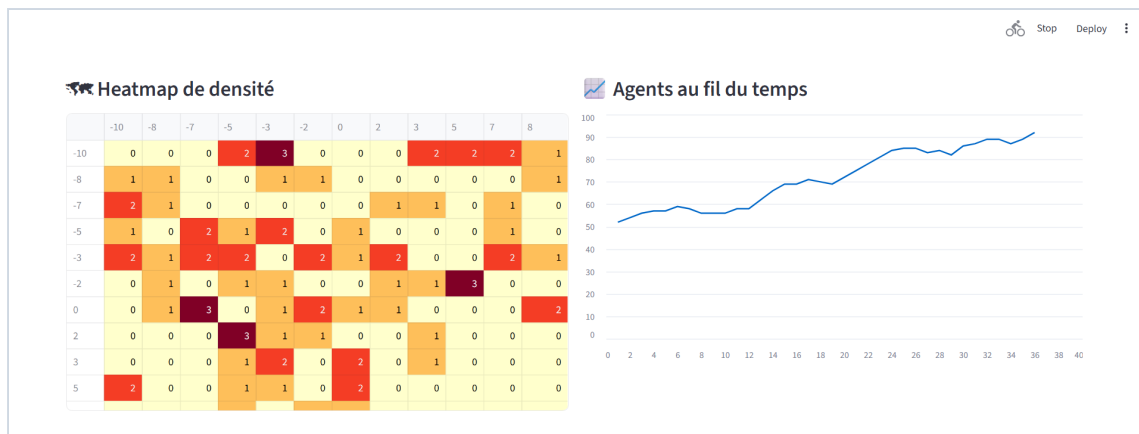


Figure 4.13 — Heatmap de densité spatiale et courbe d'évolution du nombre d'agents au fil du temps.

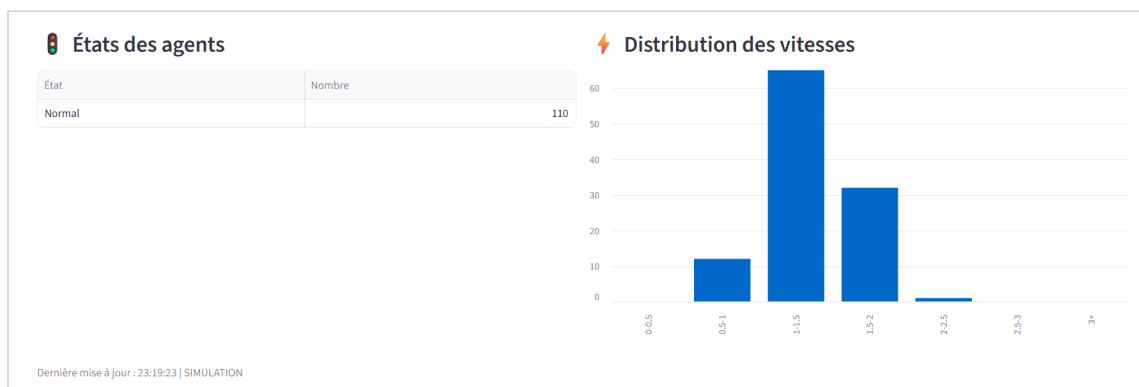


Figure 4.14 — Répartition des états des agents (110 « Normal ») et histogramme de distribution des vitesses.

4.4 — Maryem Mouslim

Réalité Virtuelle & Interaction immersive

branche : feature/vr-ux

Maryem a conçu la couche d'immersion : la configuration VR pour le Meta Quest 2, l'interaction physique avec la scène, la restitution visuelle des zones de danger, et un mode PC de repli sans casque.

a) Placement interactif de barrières

En VR, l'utilisateur pointe avec le `XRRayInteractor` et matérialise une barrière à l'endroit visé (`BarrierPlacer.cs`), permettant de canaliser dynamiquement la foule.

b) Overlay de danger

Le composant `VR0verlay.cs` affiche une zone rouge pulsée lorsqu'un danger est signalé, l'opacité oscillant via une fonction sinusoïdale. L'overlay est connecté à `ML-Client.dangerDetected`.

```
// VROverlay.cs – pulsation visuelle de la zone de danger
float a = 0.25f + 0.15f * Mathf.Sin(Time.time * 5f);
Color c = rend.material.color; c.a = a; rend.material.color = c;
```

c) Configuration XR et mode PC

Maryem a mis en place l'XR **Origin** (VR) compatible OpenXR / Meta Quest 2, testé la compatibilité du casque, et prévu un mode PC reposant sur une caméra isométrique top-down automatique lorsque aucun périphérique XR n'est actif.



Figure 4.15 — Vue VR (mode jeu) : zone de danger rouge au centre, barrière placée interactivement, foule répartie.

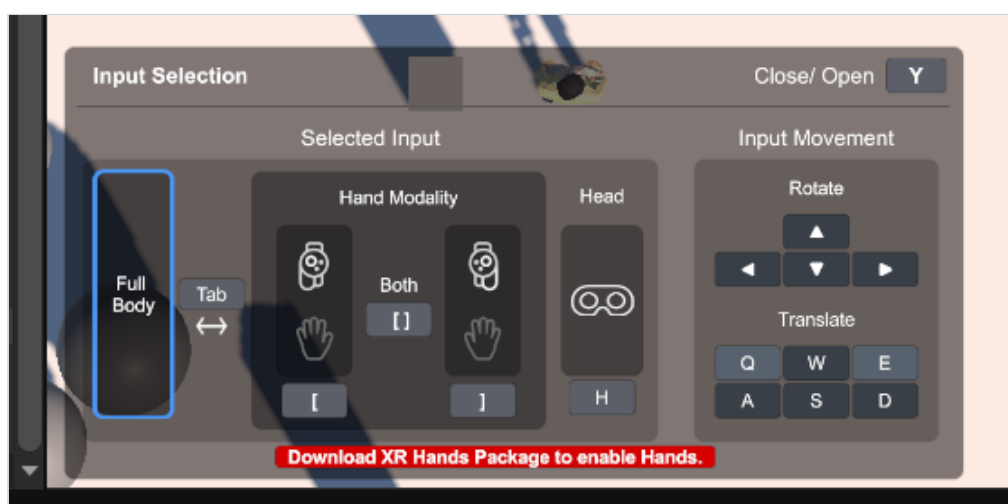


Figure 4.16 — Panneau de sélection des entrées VR : modalité des mains, tête et déplacement.

4.5 — Assil Nagda

Réseau, Multijoueur & Intégration finale

branche : feature/networking

Assil a assuré la couche réseau et le travail d'intégration finale qui réunit les contributions de toute l'équipe dans une scène unique cohérente.

a) Synchronisation réseau

S'appuyant sur Netcode for GameObjects, `NetworkSync.cs` partage l'état de danger global entre les clients via une `NetworkVariable`. Lorsque le serveur passe le danger à vrai, un `ClientRpc` déclenche l'évacuation sur tous les clients.

```
// NetworkSync.cs – danger global synchronisé + RPC d'évacuation
[ServerRpc(RequireOwnership=false)]
public void SetDangerServerRpc(bool danger) { GlobalDanger.Value = danger; }
[ClientRpc] public void TriggerEvacuationClientRpc() { ... }
```

b) Contrôleur Desktop et intégration

Pour la collaboration asymétrique (un joueur VR et un opérateur PC), `DesktopController.cs` détecte l'absence de casque, installe une caméra top-down et gère les commandes clavier (déplacement, évacuation, panique). Assil a fusionné les branches dans la scène finale, ajouté les objets `NetworkManager` et `NetworkSync`, configuré l'URP et corrigé les matériaux.

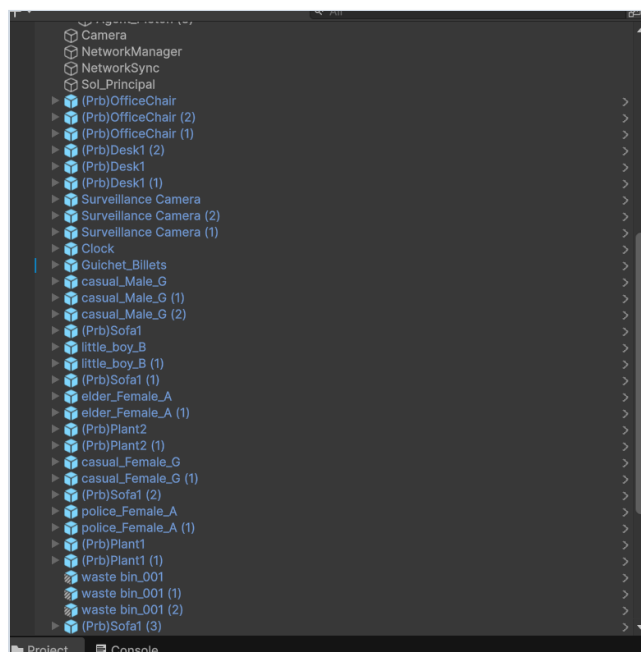


Figure 4.17 — Hiérarchie finale intégrée : `NetworkManager`, `NetworkSync` et l'ensemble des prefabs de la gare.

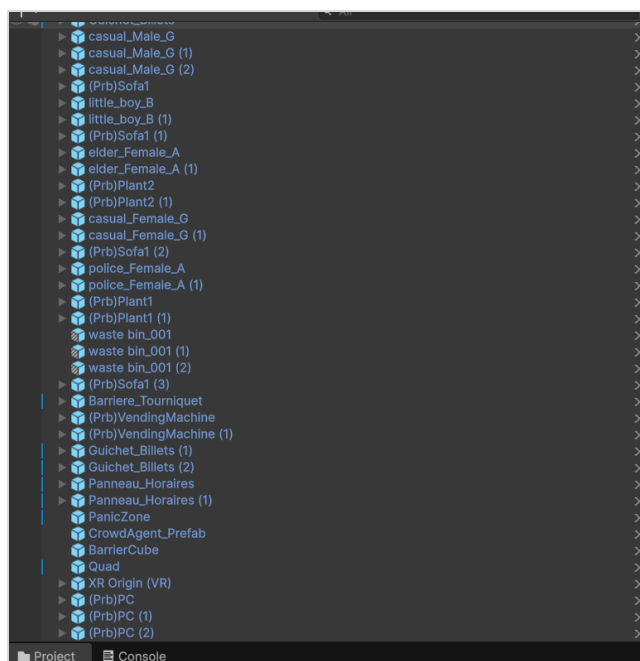


Figure 4.18 — Suite de la hiérarchie : *Barriere_Tourniquet*, *PanicZone*, *XR Origin (VR)* et *CrowdAgent_Prefab*.

4.6 — Hasnaa Saadi

Comportements collectifs & Dynamique de foule

branche : feature/panic-behavior

Hasnaa a apporté le réalisme comportemental de la foule en implémentant des modèles scientifiques reconnus de dynamique piétonne, superposés à la navigation NavMesh.

a) Modèle de force sociale

Implémentation du modèle de **Helbing & Molnár (1995)** dans `SocialForce.cs` : chaque agent subit une force de répulsion exponentielle de ses voisins proches, reproduisant le maintien d'une distance interpersonnelle.

```
// SocialForce.cs – force de répulsion de Helbing (A = 2000, B = 0.08)
Vector3 diff = transform.position - col.transform.position;
float dist = Mathf.Max(diff.magnitude, 0.01f);
force += (A * Mathf.Exp(-dist / B)) * diff.normalized;
```

b) Boids et propagation de panique

La règle de séparation des **Boids** (`BoidsBehavior.cs`) complète les forces sociales en écartant les voisins immédiats. Une **PanicZone** (`PanicTrigger.cs`) propage l'état de panique par ondes successives (rayon croissant de 5 à 35 m), modélisant la contagion comportementale.



Figure 4.19 — Gros plan sur les agents : la séparation Boids et les forces sociales maintiennent un espacement réaliste.

CHAPITRE 5

Intelligence Artificielle

L'intelligence artificielle de CrowdFlow VR combine trois approches complémentaires : un modèle d'apprentissage pour détecter les anomalies, un système de navigation pour le déplacement, et des modèles physiques pour la dynamique collective.

5.1 Détection d'anomalies — IsolationForest

Le modèle a été entraîné sur des trajectoires piétonnes réelles issues des jeux de données ETH (Zürich) et UCY (Chypre), regroupant les scènes *seq_eth*, *seq_hotel*, *zara01* et *zara02*. Au total, 4534 frames réelles décrivant 360 piétons ont permis d'isoler 227 configurations atypiques (5 %).

Nombre de frames : 1448								
Nombre de piétons : 360								
Aperçu des données :								
	frame	ped_id	x	y	vx	vy	ax	ay
0	780.0	1.0	8.456844	0.0	3.588066	1.671714	0.0	0.176292
1	786.0	1.0	9.125530	0.0	3.658583	1.662877	0.0	0.326723
2	792.0	1.0	9.787146	0.0	3.849445	1.683334	0.0	0.371084
3	798.0	1.0	10.472197	0.0	3.955450	1.598568	0.0	0.264795
4	804.0	1.0	11.066000	0.0	4.061280	1.574526	0.0	0.456390
5	804.0	2.0	13.017548	0.0	5.782591	-2.324444	0.0	-0.076606
6	810.0	1.0	11.731818	0.0	4.320563	1.644127	0.0	0.544391
7	810.0	2.0	12.087770	0.0	5.751949	-1.589052	0.0	-0.065971
8	816.0	1.0	12.381302	0.0	4.496793	1.623709	0.0	0.440576
9	816.0	2.0	11.746306	0.0	5.729815	-1.140766	0.0	0.105373
Vitesse moyenne : 5.55 m/s								
Vitesse max : 13.29 m/s								

Figure 5.1 — Sortie d'exploration des données réelles ETH : 1448 frames pour la scène *seq_eth*, 360 piétons, calcul des vitesses.

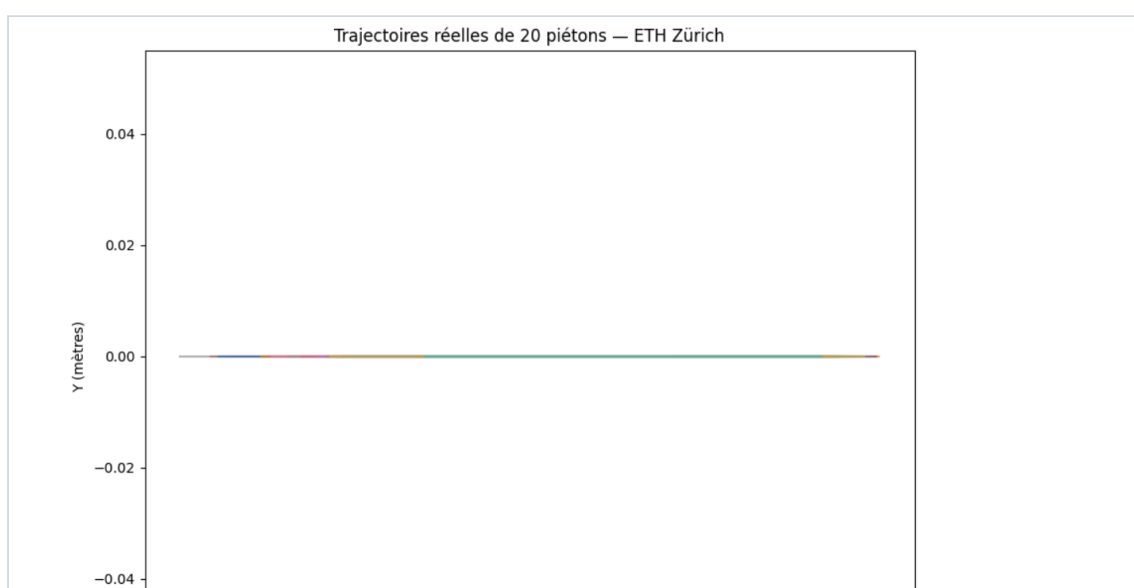


Figure 5.2 — Tracé des trajectoires réelles de piétons (ETH Zürich) reconstituées depuis le jeu de données.

Paramètre	Valeur
Algorithme	IsolationForest (apprentissage non supervisé)
Contamination	0.05 (5 % d'anomalies attendues)
Estimateurs	100 (random_state = 42)
Normalisation	StandardScaler
Caractéristiques	nombre d'agents, vitesse moyenne, écart-type, 90 ^e percentile
Communication	UDP, port 5005, format JSON, cycle 500 ms

Tableau 5.1 — Paramètres du modèle de détection d'anomalies.

5.2 Navigation autonome — NavMesh

Le pathfinding repose sur le système NavMesh d'Unity : la surface navigable de la gare est *bakée*, et chaque agent calcule son chemin vers une destination en évitant murs, colonnes et mobilier. Les vitesses s'adaptent à l'état comportemental de l'agent.

5.3 Dynamique collective — Social Force Model

Le modèle de Helbing & Molnár (1995) superpose aux trajectoires NavMesh des forces de répulsion inter-piétons, complétées par la règle de séparation des Boids. L'ensemble produit des phénomènes émergents réalistes : files, contournements et propagation de panique.

Synergie — les trois approches collaborent en temps réel : NavMesh fournit le déplacement, le Social Force Model le réalisme local, et l'IsolationForest la couche de vigilance qui peut faire basculer toute la foule en panique.

CHAPITRE 6

Big Data et tableau de bord

La chaîne Big Data relie la simulation au tableau de bord. Côté Unity, le module de journalisation exporte en continu un fichier CSV horodaté de l'état complet de la foule. Côté analyse, le jeu de données ETH/UCY traité sert de référence, et les fichiers de session alimentent les visualisations interactives.

Étape	Description
Capture	Export CSV toutes les 500 ms (frame, ped_id, x, z, speed, state)
Stockage	Fichiers session_HH:mm:ss.csv dans le dossier persistant Unity
Traitement	Agrégation pandas, calcul de densité, états et vitesses
Restitution	Visualisations Streamlit temps réel (rafraîchissement continu)

Tableau 6.1 — Étapes de la chaîne de traitement des données.

Le serveur d'inférence Python affiche en continu, pour chaque cycle reçu d'Unity, le nombre d'agents, la vitesse moyenne et le statut de danger, confirmant le bon fonctionnement de la boucle temps réel.

[illegible]

Figure 6.1 — Sortie du serveur d'inférence Python en fonctionnement : flux temps réel « Agents / Vitesse moyenne / Danger ».

CHAPITRE 7

Architecture et interaction en réalité virtuelle

7.1 Configuration XR / OpenXR

L'immersion repose sur un `XR Origin (VR)` configuré pour le Meta Quest 2 via OpenXR. L'utilisateur observe la gare à l'échelle humaine et place des barrières par pointage laser, tandis qu'un overlay rouge pulsé matérialise les zones de danger signalées par le moteur ML.

7.2 Mode PC sans casque

Pour la portabilité et la démonstration, un mode PC bascule automatiquement sur une caméra isométrique top-down lorsque aucun périphérique XR n'est détecté, avec contrôles clavier pour le déplacement, l'évacuation et la panique.

7.3 Multijoueur asymétrique

Grâce à Netcode for GameObjects, un joueur immergé en VR et un opérateur sur PC partagent la même simulation : l'état de danger global est synchronisé entre clients, et toute évacuation déclenchée se propage instantanément.

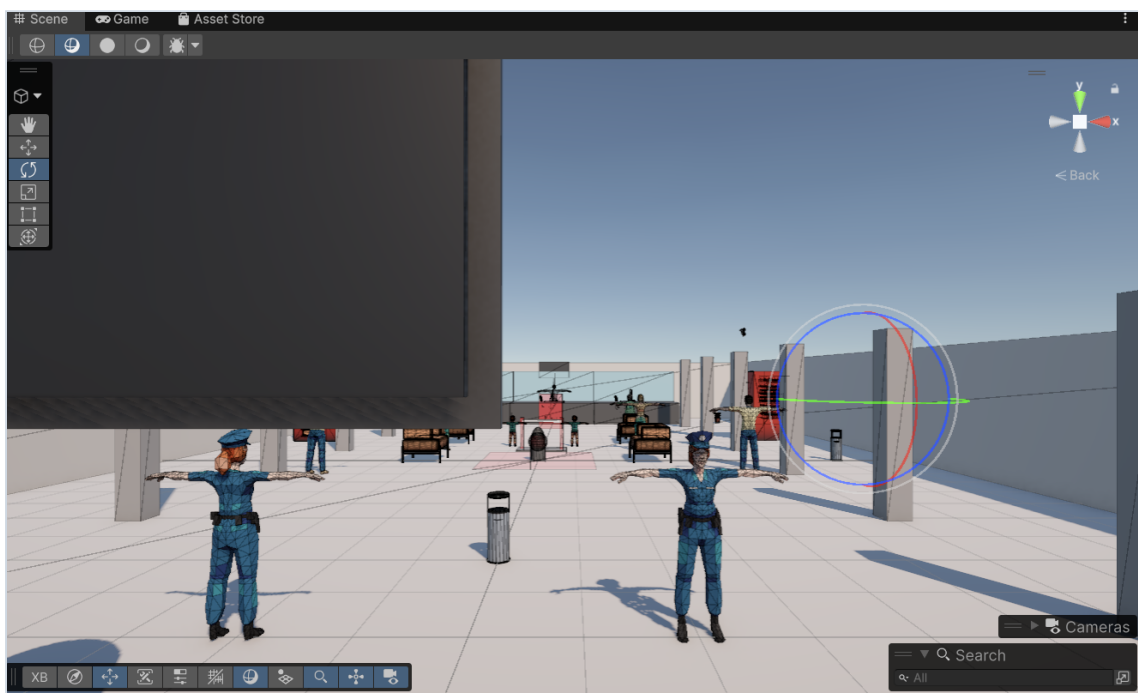


Figure 7.1 — Rendu de la scène avec personnages variés (dont agents de police) et gizmo d'interaction VR.

CHAPITRE 8

Gestion de projet et organisation GitHub

Le projet a adopté un workflow Git collaboratif rigoureux : une branche `master` stable, une branche `develop` d'intégration, et une branche `feature` par membre, fusionnées via Pull Requests. Cette organisation a permis à six contributrices de travailler en parallèle sans collision. Au total, le dépôt comptabilise soixante-et-un commits répartis sur neuf branches.

Branche	Membre	Domaine
<code>master / develop</code>	Équipe	Intégration stable
<code>feature/ml-engine</code>	Maroua	Modèle ML & prédiction
<code>feature/navmesh-ai</code>	Imane	NavMesh & pathfinding
<code>feature/data-pipeline</code>	Hoda	Big Data & dashboard
<code>feature/vr-ux</code>	Maryem	VR & interaction
<code>feature/panic-behavior</code>	Hasnaa	Comportements & panique
<code>feature/networking</code>	Assil	Réseau & multijoueur
<code>feature/scene-assets-maroua</code>	Maroua	Assets & scène 3D

Tableau 8.1 — Branches de fonctionnalités et domaines associés.

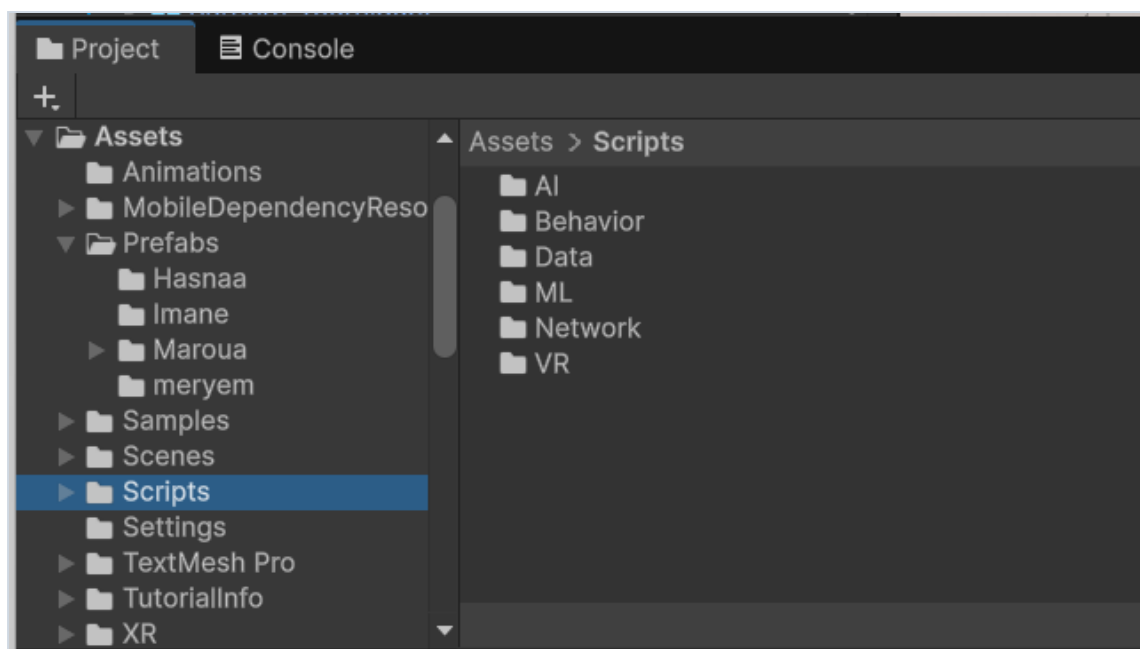


Figure 8.1 — Organisation miroir dans le projet Unity : un dossier de scripts par domaine, aligné sur les branches Git.

CHAPITRE 9

Résultats, tests et performances

Le système fonctionne de bout en bout : la foule navigue, le serveur ML reçoit les flux et signale le danger, le tableau de bord affiche les métriques en temps réel, et l'évacuation se déclenche correctement, en VR comme en mode PC.

Composant	Test réalisé	Résultat
Modèle ML	Entraînement sur 4534 frames ETH/UCY	227 anomalies détectées (5 %)
Serveur UDP	Réception du flux Unity sur le port 5005	Inférence temps réel opérationnelle
NavMesh	Navigation de 1, 10 puis 50 agents	0 erreur · $\approx$ 19–27 FPS à 50 agents
Évacuation	Déclenchement par la touche E	Tous les agents vers la sortie d'urgence
Dashboard	Affichage live de 70 agents à 1,43 m/s	Heatmap et métriques cohérentes
VR / PC	Bascule casque ↔ caméra top-down	Détection XR automatique

Tableau 9.1 — Synthèse des tests réalisés et de leurs résultats.

```
PS C:\Users\marwa_litxmy9\Downloads\crowdflowvr> cd C:\Users\marwa_litxmy9\Downloads\crowdflowvr\Python\ml_engine
PS C:\Users\marwa_litxmy9\Downloads\crowdflowvr\Python\ml_engine> python infer_server.py
=== ML Server CrowdFlow VR ===
En attente de donnees Unity sur port 5005...
```

Figure 9.1 — Lancement du serveur ML : « En attente de données Unity sur port 5005 » — la chaîne est prête à recevoir la simulation.

CONCLUSION

Conclusion générale et perspectives

CrowdFlow VR démontre qu'il est possible de réunir, dans un projet étudiant cohérent, une simulation comportementale d'agents autonomes, un moteur de Machine Learning entraîné sur des données piétonnes réelles, une chaîne Big Data temps réel et une interface de réalité virtuelle immersive — le tout orchestré par une organisation Git collaborative.

Chacune des six contributrices a apporté un pilier indispensable : la scène et l'IA (Maroua), la navigation (Imane), la donnée et le tableau de bord (Hoda), l'immersion VR (Maryem), le réseau et l'intégration (Assil), et la dynamique collective (Hasnaa). L'assemblage de ces briques produit un système fonctionnel et démontrable, répondant aux objectifs initialement fixés.

Perspectives

- Optimiser le rendu au-delà de 50 agents (instancing GPU, niveaux de détail) afin d'atteindre des foules plus denses à fréquence d'images stable ;
- Remplacer la règle de danger par seuils par une décision pleinement issue du modèle ML embarqué ;
- Enrichir le multijoueur (plusieurs opérateurs, rôles différenciés) et persister les sessions pour analyse a posteriori ;
- Étendre les scénarios (incendie, sorties bloquées) et calibrer la propagation de panique sur des études de cas réelles.

BIBLIOGRAPHIE

Références bibliographiques

#	Référence
[1]	Lerner, A., Chrysanthou, Y., & Lischinski, D. (2007). <i>Crowds by Example</i> . Computer Graphics Forum (ICCV 2007). — jeu de données UCY.
[2]	Pellegrini, S., Ess, A., Schindler, K., & Van Gool, L. (2009). <i>You'll Never Walk Alone: Modeling Social Behavior for Multi-target Tracking</i> . ICCV 2009. — jeu de données ETH.
[3]	Helbing, D., & Molnár, P. (1995). <i>Social Force Model for Pedestrian Dynamics</i> . Physical Review E, 51(5), 4282–4286.
[4]	Reynolds, C. W. (1987). <i>Flocks, Herds and Schools: A Distributed Behavioral Model (Boids)</i> . SIGGRAPH '87.
[5]	Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). <i>Isolation Forest</i> . IEEE ICDM 2008.
[6]	OpenTraj — Human Trajectory Prediction Dataset Benchmark. github.com/crowdbotp/OpenTraj (source des données ETH/UCY).
[7]	Unity Technologies. <i>Unity 6 Documentation — NavMesh, XR Interaction Toolkit, Netcode for GameObjects</i> .

CrowdFlow VR — Simulation de Gestion de Foule par IA en Réalité Virtuelle · 2025/2026
 ENSA Berrechid — Université Hassan 1^{er} · Module Ingénierie et Maquette Numérique · Encadré par
 Prof. Hamid Hrimech